

Projet : PetSitter – Mise en relation Gardiens / Propriétaires d'Animaux

Durée : 4h en classe (lancement) + 2 semaines en autonomie | Équipe : 3 étudiants | Techno : PHP (POO), MySQL, Architecture MVC

1. Présentation du projet

La plateforme associative "PetSitter Local" met en relation des propriétaires d'animaux qui cherchent une garde ponctuelle (week-end, vacances, déplacement professionnel) et des habitants volontaires pour garder les animaux du quartier. Aujourd'hui les annonces circulent sur un groupe de messagerie, sans suivi, sans garantie qu'une seule personne s'engage sur une période, et sans aucun moyen de savoir si un gardien est fiable.

L'objectif : développer une application web permettant aux propriétaires de publier une demande de garde, aux gardiens de candidater et d'être validés, puis — une fois la garde terminée — à chacun de noter l'autre pour construire une réputation de confiance sur la plateforme.

2. Spécifications Fonctionnelles (Répartition par Étudiant)

Étudiant A : Le Module "Propriétaire" (Gestion des annonces)

- **Publier une annonce** : formulaire de création (titre, type d'animal, ville, dates de garde, description des besoins).
- **Consulter les candidatures** : liste des gardiens ayant postulé à une annonce, avec leur message de motivation et leur note moyenne actuelle.
- **Valider un gardien** : choix d'un candidat parmi la liste ; l'annonce passe au statut "Pourvue" et les autres candidatures sont automatiquement refusées.
- **Mes annonces** : historique des annonces publiées avec leur statut (Ouvverte, Pourvue, Terminée).
- **Back-office associé** : statistiques du propriétaire (nombre de gardes confiées, gardien le plus sollicité).

Étudiant B : Le Module "Gardien" (Recherche et candidature)

- **Consulter les annonces disponibles** : liste filtrable par ville et par type d'animal, uniquement les annonces au statut "Ouvverte".
- **Postuler** : envoi d'une candidature avec un court message de motivation ; impossible de postuler deux fois à la même annonce.
- **Mes candidatures** : suivi de l'état de chaque candidature (En attente, Acceptée, Refusée).
- **Mon profil public** : page affichant le nom du gardien, sa note moyenne et le nombre de gardes réalisées.

Étudiant C : Le Module "Avis & Réputation" (Back-office Admin)

- **Clôture de garde** : lorsqu'une annonce "Pourvue" atteint sa date de fin, elle passe automatiquement (ou manuellement) au statut "Terminée", ce qui débloque la notation.
- **Laisser un avis** : une fois la garde "Terminée", le propriétaire et le gardien peuvent chacun noter l'autre (note de 1 à 5 + commentaire).
- **Calcul de la réputation** : calcul et affichage de la note moyenne d'un utilisateur à partir de tous les avis reçus.
- **Espace Admin** : modération des avis (suppression d'un commentaire abusif), gestion des comptes, statistiques globales (nombre de gardes réalisées, note moyenne de la plateforme).

3. Spécifications Techniques & Architecture

Le Socle Commun

Les trois étudiants doivent concevoir ensemble l'architecture dès les premières heures :

- **Un routeur unique** : qui aiguille les requêtes vers les contrôleurs respectifs (AnnonceController, CandidatureController, AvisController).
- **Une classe Utilisateur commune** : propriétés (id, nom, email, password) ; un même utilisateur peut être propriétaire et gardien selon les annonces.
- **Une connexion PDO partagée** et une gestion de session commune pour l'authentification.

Classes principales

- **Classe Annonce** : propriétés (id, titre, type_animal, ville, date_debut, date_fin, description, statut, id_proprietaire). Méthode cloturer(\$idCandidatureRetenue) qui valide un gardien et refuse automatiquement les autres candidatures.
- **Classe Candidature** : propriétés (id, message, statut, date_candidature, id_annonce, id_gardien).
- **Classe Avis** : propriétés (id, note, commentaire, date, id_annonce, id_auteur, id_cible). Méthode statique Avis::noteMoyenne(\$idUtilisateur) qui calcule la moyenne des notes reçues par un utilisateur.

Base de données (Schéma suggéré)

- utilisateurs : id, nom, email, password, role.
- annonces : id, titre, type_animal, ville, date_debut, date_fin, description, statut, #id_proprietaire.
- candidatures : id, message, statut, date_candidature, #id_annonce, #id_gardien.
- avis : id, note, commentaire, date, #id_annonce, #id_auteur, #id_cible.

4. Planning de Charge de Travail

Séance de lancement en classe (4h)

- 30 min — Lecture du sujet en équipe et répartition des rôles A / B / C.
- 1h — Conception commune : MCD/MLD et diagramme de classes (les 4 tables, les statuts, le cycle de vie d'une annonce).
- 45 min — Mise en place du socle commun : création de la BDD, classe PDO, routeur, authentification partagée.
- 1h45 — Démarrage individuel : Étudiant A commence le CRUD des annonces, Étudiant B la liste filtrée des annonces disponibles, Étudiant C la structure de la table avis et le squelette du contrôleur.

2 semaines en autonomie

- Semaine 1 — Développement des modules individuels : publication d'annonces et gestion des candidatures (A), recherche et candidature (B), page de clôture de garde et formulaire de notation (C).
- Semaine 2 — Intégration des trois modules, calcul et affichage de la note moyenne sur les profils, sécurisation (hash, requêtes préparées, contrôle d'accès), fusion Git, finitions et documentation.

5. Livrables et Contraintes

Livrables

- Code source : structure MVC propre, dépôt Git avec un historique de commits par étudiant.
- Base de données : script .sql avec un jeu de données réaliste (au moins 3 annonces, 4 candidatures et 3 avis dans des états variés).
- Documentation courte : schéma relationnel (MLD), diagramme de classes, et explication de la logique de clôture d'annonce et de calcul de la note moyenne.

Contraintes

- **Règle métier** : un gardien ne peut pas postuler deux fois à la même annonce.
- **Règle métier** : un propriétaire ne peut valider qu'un seul gardien par annonce ; les autres candidatures sont automatiquement refusées.
- **Règle métier** : un avis ne peut être laissé que sur une garde au statut "Terminée", et chaque utilisateur ne peut noter qu'une seule fois par garde.
- **Sécurité** : password_hash() / password_verify() et requêtes préparées PDO obligatoires.
- **Accès** : un utilisateur ne doit voir que ses propres annonces et candidatures ; seul l'admin accède à la modération des avis.

6. Critères d'Évaluation

Élément évalué	Type	Barème /20
Architecture commune & MVC (routeur, socle, BDD)	Collectif	3 pts
Gestion de Git & intégration (commits réguliers, fusion propre)	Collectif	2 pts
Module individuel (A, B ou C) : CRUD, respect du cahier des charges, code POO	Individuel	7 pts
Logique métier (clôture d'annonce, refus auto, calcul de note moyenne)	Individuel	4 pts
Sécurité & robustesse (injections SQL, hachage, contrôle des accès)	Individuel	3 pts
UX & gestion des erreurs	Individuel	1 pt